Academia

A unified school management platform — CS50 C Project

Project Overview

Academia is an all-in-one school management platform that keeps academic records, communication and everyday school tasks in one place. It is written mostly in C, with SQL for the database and HTML, CSS and JavaScript for the interface. Grades, attendance tracking, school announcements and student profiles all live in a single system, so students, parents, teachers and administrators do not have to juggle several tools.

Setup is simple: one launch script builds and starts the server and opens the site in the browser. After sign-in, each user gets a dashboard for their role. Students and parents can check grades, attendance, assignments and school news. Teachers and administrators get one central place to update records and run daily operations.

The Problem We Set Out to Solve

Schools often split records across gradebooks, attendance sheets, email threads and paper notices. Families have to hunt for information, and staff enter the same data more than once. Academia puts one source of truth behind every role, so a grade a teacher enters is the same grade a student and parent see.

Key Features

Role	What they can do
Student	View course grades, GPA on a 4.0 scale, daily attendance history, assignments and school news.
Parent	Monitor a linked child's grades, GPA, attendance logs and school notices.
Teacher / Staff	Enter or update grades (the student's GPA is recalculated automatically), record daily attendance and publish announcements.
Administrator	Central oversight of records and daily operations.

Technical Architecture

Backend (C). A standalone HTTP server built on Winsock2 serves the static pages and a REST API. The code is split into modules: authentication and session tokens, database setup, and separate student, staff and parent modules, each in its own folder.

Database (SQLite3). One relational database, academia.db, created from a schema and seed script. Tables cover users, parents, students, staff, courses, enrollments, grades, attendance, assignments and announcements, linked by foreign keys.

Frontend (HTML/CSS/JavaScript). Role-based pages for the landing screen, sign-in, and the student, staff and parent portals, plus dedicated error pages (403, 429, 500, 504, session expired, account locked, link expired, offline, maintenance). A shared stylesheet follows the Academia brand guide.

Endpoint	Purpose
POST /api/login	Checks credentials against the user table and starts a session.
GET /api/student-data	Student record, courses, grades, attendance and announcements.
GET /api/parent-data	Linked child's information, grades and attendance.
GET /api/staff-data	Staff courses, enrolled students, grade records and attendance history.
POST /api/staff/grade	Inserts or updates a grade and recalculates the student's GPA.
POST /api/staff/attendance	Logs attendance.
POST /api/staff/announcement	Publishes a new announcement.

How a Request Flows

The browser sends a request carrying a session token. The server looks the token up in its session table and rejects it if it is missing (401) or belongs to the wrong role (403). The user ID comes from the server-side session, never from the URL or request body, so a student can only load their own data. The role module then reads SQLite and returns the result to the page. If a page is opened directly from disk, the frontend falls back to exported state files and demo accounts so the interface still loads.

Design and Branding

The interface follows the Academia brand sheet: deep navy (#0B3D91) for headers, royal blue (#2563EB) for buttons and links, sky blue (#60A5FA) for accents, golden yellow (#FACC15) for highlights, and cool grays for surfaces. The typeface is Poppins. The motto is *Education Today. A Brighter Tomorrow.*

Running the Project

Run run_server.bat, or start academia_server.exe directly. The server listens on port 3000 and opens http://localhost:3000. Demo accounts (password: password123): student1 (Alex Rivera), parent1 (Marcus Rivera), staff1 (Dr. Eleanor Vance).

What Each of Us Did

Team member	Contribution
Harshul Gupta	Co-led the project timeline and overall structure. Planned development milestones, designed how the system's parts fit together, and integrated the main features so everything ran as one cohesive app.
Dmitrii Buianov	Co-led the project timeline and core programming. Built the foundational parts

	of the system, organized the development stages, and kept technical goals on schedule.
Lawrence Mao	Feature development and backend logic. Turned the initial ideas and technical plans into the core code behind the app's main functions and user interactions.
Seungho Kim	Testing and quality assurance. Tested the platform across user roles, tracked down bugs and edge cases, and confirmed the whole user flow worked smoothly before completion.

Outcome and Future Work

The result is a working full-stack school portal with real read/write operations on a shared database. Possible next steps: password hashing, an administrator dashboard with user management, messaging between parents and teachers, and cross-platform support beyond Windows.

Team and Roles at a Glance

Academia was built by a team of four for the CS50 final project. Two of us shared project leadership, one focused on backend feature code, and one owned quality assurance. Splitting the work this way meant planning, building and testing could move in parallel instead of waiting on each other.

Goals

We set four goals at the start:

- **One place for everything.** Grades, attendance, announcements and profiles in a single system.
- **Role-appropriate views.** Every user sees what matters to them and nothing they should not.
- **Easy to run.** A single launch script, no heavy third-party dependencies.
- **Reliable.** Every user flow works from sign-in to sign-out, and the interface never crashes, even when the server is unavailable.

Design Approach

We chose a deliberately plain, procedural style of C. The server uses the standard Windows libraries (Winsock2 for networking and the built-in SQLite3 library for data) and standard file I/O to serve pages. Because there are no bulky dependencies, the project compiles with the standard Microsoft compiler in one command. The tradeoff is that we wrote pieces, such as the HTTP handling and JSON output, ourselves, which taught us a great deal about how a web server works underneath a framework.

Backend Modules in Detail

The backend is organized so each file has one job. This kept the code readable and let teammates work on different modules without stepping on each other.

Module	Responsibility

Academia_Server.c	The HTTP server. Listens on port 3000, serves HTML, CSS, JavaScript and images, reads the session token from each request, and routes every /api call to the right module.
Academia_Auth.c	Sign-in and sessions. Checks the username, password and role against the same database row, creates a session token, and keeps a small in-memory session table. Supports logging out a single session or all sessions.
Academia_Database.c	Opens academia.db, turns on foreign-key enforcement, creates the tables if they are missing, and seeds the demo data.
Academia_Student.c	Gathers everything the Student Portal needs: profile, enrolled courses, grades, attendance, assignments and announcements.
Academia_Staff.c	Staff data, plus the actions teachers take: recording grades, taking attendance and publishing announcements.
Academia_Parents.c	The parent view: the parent's linked child and that child's grades and attendance.

Authentication and Sessions

Signing in requires a username, a password and a role, and all three must match one row in the user table. Matching on the role means a student account cannot be used to open the staff portal. On success the server creates a session token and returns the user's id, name, username and role to the browser, which sends the token with later requests.

Sessions live in memory in a fixed-size table. When the table is full, the oldest session is dropped to make room for the newest. A request with no token or an unknown token is answered with 401, and a valid token used on the wrong role's endpoint is answered with 403. The user id always comes from the session on the server, never from the URL or request body, which is why a student can only ever load their own data.

All database lookups use prepared statements with bound parameters instead of building SQL from user text, which prevents SQL injection. Text placed into JSON replies is cleaned so a stray quotation mark or backslash cannot break the response.

The Database

Academia stores everything in one SQLite3 file. Using foreign keys keeps records consistent: deleting a user, for example, automatically removes that user's student, parent or staff profile.

Table	What it stores
user	Username, password, first and last name, and role (student, parent or staff).
student	Grade level, GPA and a link to the student's parent.
parent / staff	Contact phone number, linked to the user record.
course	Course name and the staff member who teaches it.
enrollment	Which students are in which courses, and whether the enrollment is active.
grade	A score from 0 to 100 for a student in a course.

attendance	Present or absent, per student and course.
assignment	Assignment name and due date for a course.
announcement	Title, message and date of a school notice.

Demo Data

The database ships with a small seed set so the system can be tried immediately: a student (Alex Rivera, grade 11), that student's parent (Marcus Rivera), a teacher (Dr. Eleanor Vance) and three courses: Complete HTML Tutorial, Complete CSS Tutorial and Complete JavaScript Tutorial.

How Data Reaches the Browser

When a student opens their dashboard, the server confirms the session, then the Student module runs a handful of queries and writes the results as a JavaScript state file. The server returns that file to the page, which reads it and fills in grades, attendance, assignments and news. The same pattern is used for the staff and parent dashboards. Because the same state files are kept on disk, the frontend can still display data when a page is opened without the server running.

The Three Portals

Student Portal

Students see each course with its grade, their GPA on a 4.0 scale, attendance history, upcoming assignments sorted by due date, and school announcements, newest first. Assignments are limited to the courses the student is enrolled in.

Parent Portal

Parents see their linked child's profile, grades, GPA and attendance, along with school notices. This replaces waiting for report cards or chasing emails.

Staff Portal

Teachers enter or update grades, and the student's GPA is recalculated automatically so records never drift apart. They can record daily attendance and publish announcements. These write actions go straight into the database, so students and parents see them the next time they load their dashboards.

Error and Status Pages

We also built dedicated pages for situations users do run into: access denied (403), too many requests (429), server error (500), gateway timeout (504), session expired, account locked, expired link, no internet connection and scheduled maintenance. Each uses the same brand styling so failures still feel like part of Academia.

Frontend Behavior

A shared JavaScript file acts as the client for the API. It signs the user in, stores the token, requests the right data for the current page and renders it. If the pages are opened directly from disk, it falls back to the exported state files and built-in demo accounts, so the interface never shows a blank screen.

Testing and Quality Assurance

Seungho Kim tested the platform from the point of view of each role rather than feature by feature. Walking the whole path as a student, a parent and a teacher exposed problems that testing a single function would have missed. The checks covered:

- Signing in with correct and incorrect usernames, passwords and roles, and signing out.
- Confirming each role reaches only its own portal and data.
- Entering grades and attendance as staff, then verifying the student and parent dashboards updated correctly.
- Checking GPA recalculation after grades changed.
- Edge cases such as empty fields, missing data and unexpected text in announcements.
- Opening pages with and without the server running to confirm the fallback works.

Challenges and What We Learned

Writing an HTTP server and JSON output by hand meant handling details that frameworks normally hide, such as parsing requests, setting headers and escaping text. Several modules defined helpers with the same name, which collided when everything was linked together, so we learned to keep module-private helpers static. Coordinating four people on one codebase also taught us to agree on file ownership and interfaces early.

Known Limitations

We documented the limits of the current version honestly instead of hiding them:

- Passwords are stored as plain text in the demo database and should be hashed before real use.
- Announcements are school-wide rather than filtered per course.
- The state files are shared, so the server handles one request at a time; that is safe for now but would need rework for concurrent users.
- Generated text is not fully escaped, so a quotation mark in an announcement could break a page's data.
- The server targets Windows and expects to be started from the project root.

Future Work

- Hash passwords and add stronger session handling, such as expiry times.
- Filter announcements by course and by audience.
- Build a full administrator dashboard for managing users, courses and enrollment.
- Add parent–teacher messaging.

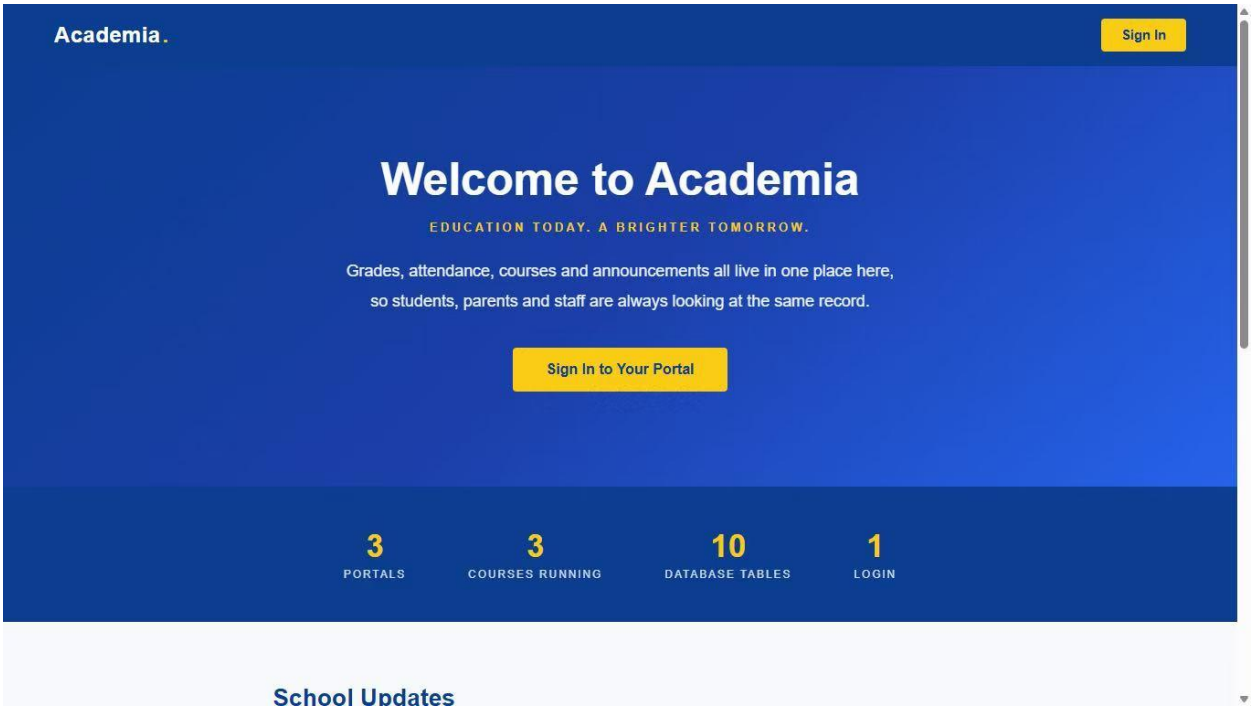
- Support concurrent users and other operating systems.
- Add automated tests so every release is checked the same way.

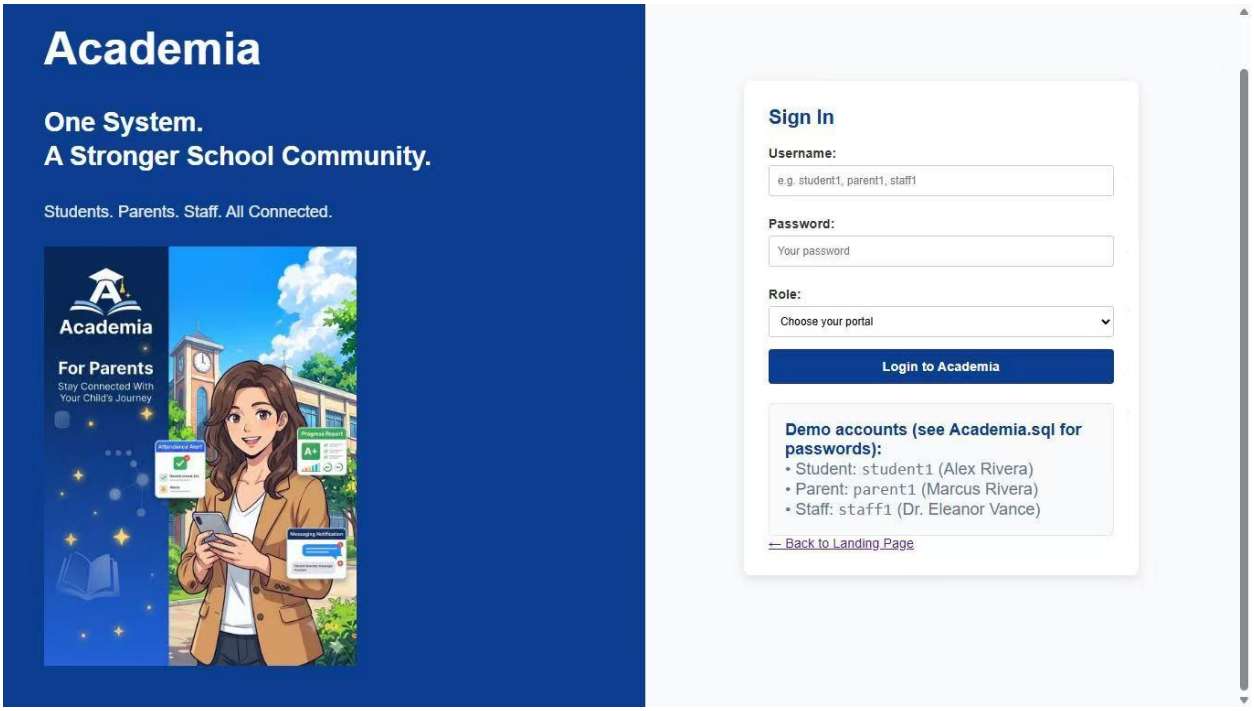
Conclusion

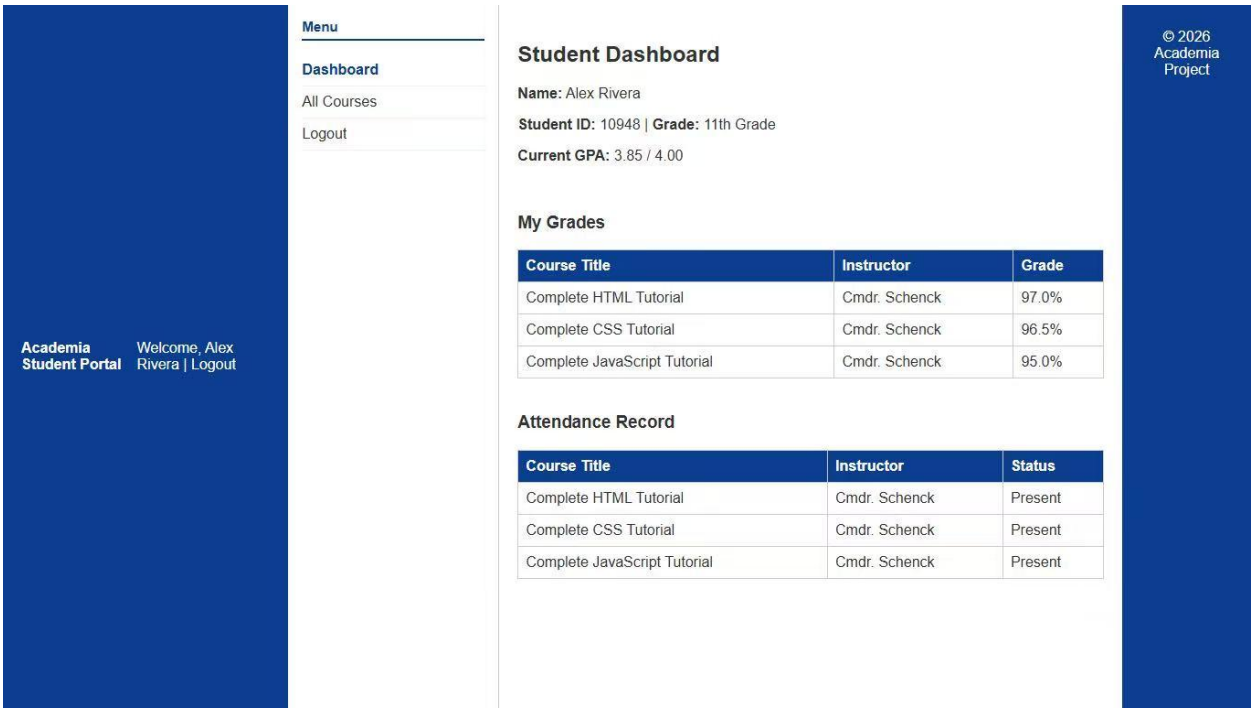
Academia shows that a small team can build a complete, role-aware school platform using plain C, SQLite and a simple web frontend. Grades, attendance, announcements and profiles now live in one place, and each person sees the view that fits them. The project brought together planning, backend programming, interface design and testing, and the team is proud of how it came together as one cohesive application.

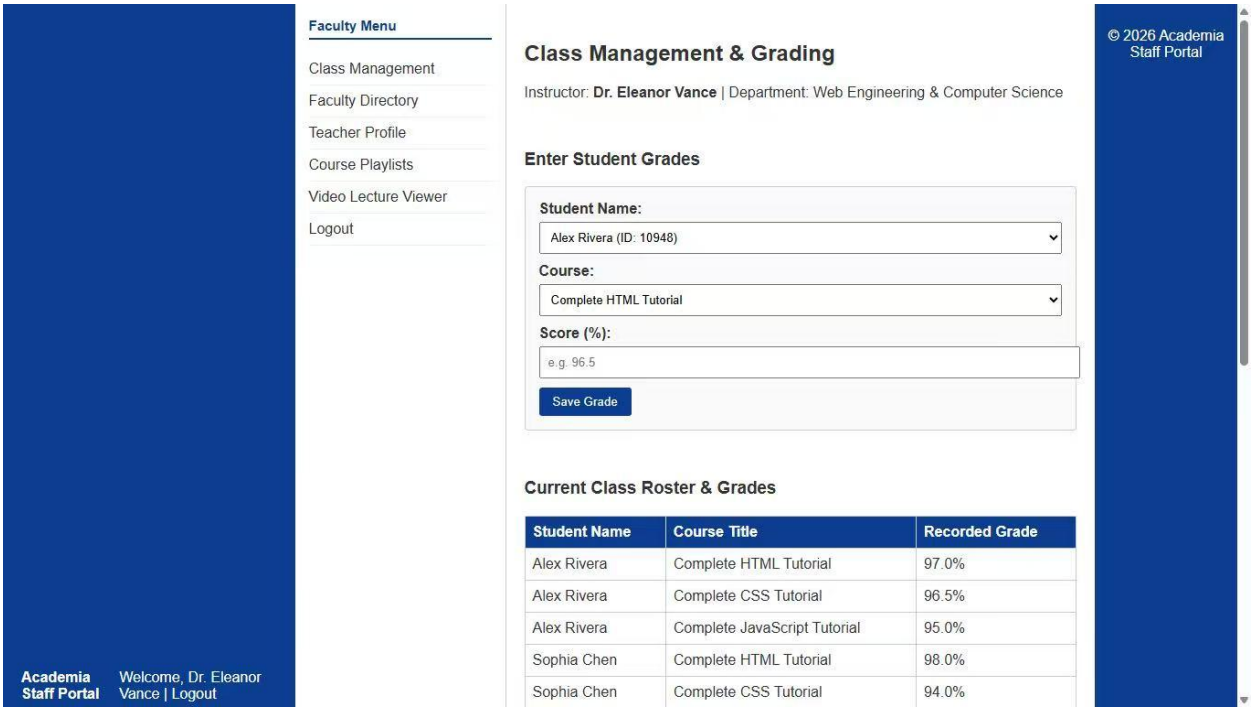
Project Details

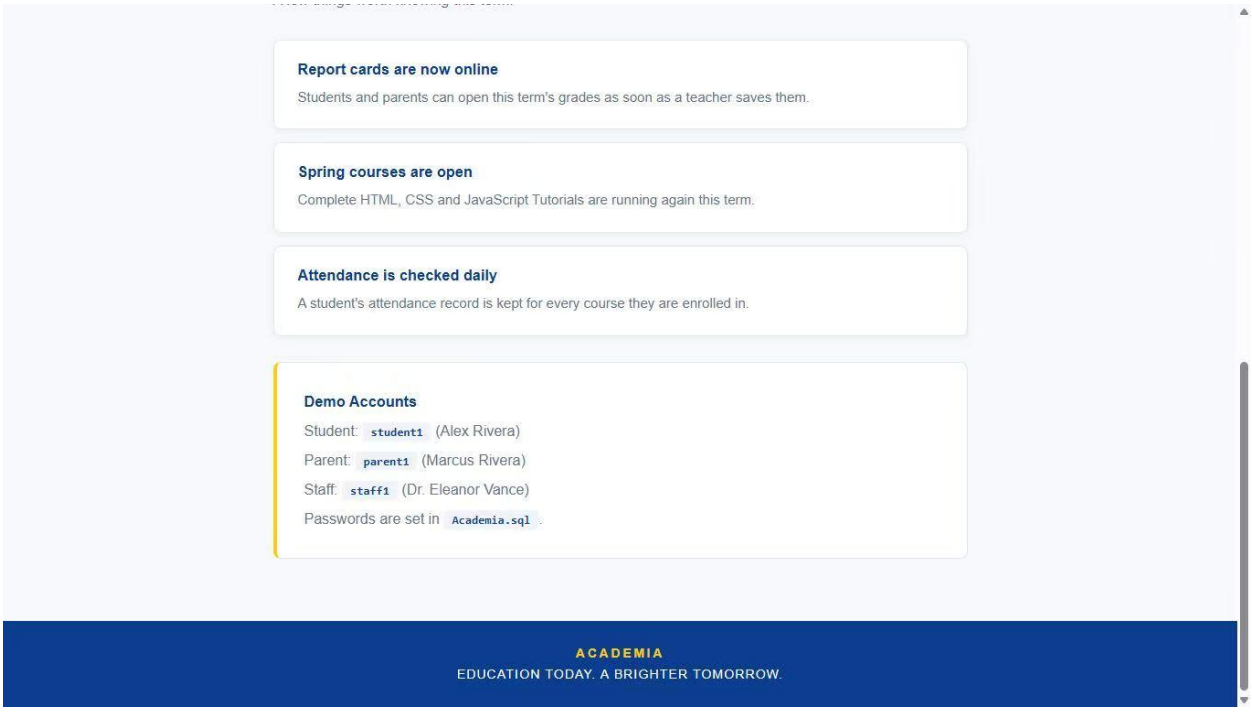
Course: CS50 Final Project
Team: Harshul Gupta, Dmitrii Buianov, Lawrence Mao, Seungho Kim
Stack: C, SQLite3, HTML, CSS, JavaScript
Run: run_server.bat, then open http://localhost:3000




Academia
One System.
A Stronger School Community.
Students. Parents. Staff. All Connected.
Academia
For Parents
Stay Connected With Your Child's Journey
Sign In
Username:
e.g. student1, parent1, staff1
Password:
Your password
Role:
Choose your portal
Login to Academia
Demo accounts (see Academia.sql for passwords):
• Student: student1 (Alex Rivera)
• Parent: parent1 (Marcus Rivera)
• Staff: staff1 (Dr. Eleanor Vance)
← Back to Landing Page


Academia Student Portal
Welcome, Alex Rivera | Logout
Menu
Dashboard
All Courses
Logout
Student Dashboard
Name: Alex Rivera
Student ID: 10948 | Grade: 11th Grade
Current GPA: 3.85 / 4.00
My Grades
Course Title | Instructor | Grade
Complete HTML Tutorial | Cmdr. Schenck | 97.0%
Complete CSS Tutorial | Cmdr. Schenck | 96.5%
Complete JavaScript Tutorial | Cmdr. Schenck | 95.0%
Attendance Record
Course Title | Instructor | Status
Complete HTML Tutorial | Cmdr. Schenck | Present
Complete CSS Tutorial | Cmdr. Schenck | Present
Complete JavaScript Tutorial | Cmdr. Schenck | Present
© 2026 Academia Project


Faculty Menu
Class Management
Faculty Directory
Teacher Profile
Course Playlists
Video Lecture Viewer
Logout
Class Management & Grading
Instructor: Dr. Eleanor Vance | Department: Web Engineering & Computer Science
Enter Student Grades
Student Name:
Alex Rivera (ID: 10948)
Course:
Complete HTML Tutorial
Score (%):
e.g. 96.5
Save Grade
Current Class Roster & Grades
Student Name
Course Title
Recorded Grade
Alex Rivera
Complete HTML Tutorial
97.0%
Alex Rivera
Complete CSS Tutorial
96.5%
Alex Rivera
Complete JavaScript Tutorial
95.0%
Sophia Chen
Complete HTML Tutorial
98.0%
Sophia Chen
Complete CSS Tutorial
94.0%
© 2026 Academia Staff Portal
Academia Staff Portal
Welcome, Dr. Eleanor Vance | Logout


Report cards are now online
Students and parents can open this term's grades as soon as a teacher saves them.
Spring courses are open
Complete HTML, CSS and JavaScript Tutorials are running again this term.
Attendance is checked daily
A student's attendance record is kept for every course they are enrolled in.
Demo Accounts
Student: student1 (Alex Rivera)
Parent: parent1 (Marcus Rivera)
Staff: staff1 (Dr. Eleanor Vance)
Passwords are set in Academia.sql .
ACADEMIA
EDUCATION TODAY. A BRIGHTER TOMORROW.